



FAKULTA
INFORMAČNÍCH
TECHNOLGIÍ
ČVUT V PRAZE

Webová aplikace pro testování úloh v Terraform

Terraform Wars

Bc. Alexandr Czerný

FIT ČVUT v Praze

Červen 2026



Terraform a IaC

Princip

- **Infrastruktura jako kód** — deklarativní popis cloudových zdrojů v jazyce HCL.
- Workflow plan → apply: Terraform porovná požadovaný a skutečný stav, aplikuje rozdíl.
- Komunikace s cloudem skrze poskytovatele (providers) a jejich API.

Multi-cloud podpora

Stejný jazyk a workflow napříč poskytovateli (AWS, GCP, Azure, ...), ale samotný kód mezi poskytovateli **přenositelný není** — každý provider má vlastní zdroje a jejich schémata.



Existující platformy

Analýza existujících řešení

- **KodeKloud** — interaktivní lekce, ale v simulovaném prostředí. Netestuje skutečný stav infrastruktury.
- **Killercoda** — sandboxové scénáře v izolovaných VM, pouze 7 úloh pro Terraform. **Žádný bezpečný způsob** napojení vlastního cloudového prostředí — uživatel musí natvrdo vepsat access key a secret key do scénáře.





Náš přístup

Terraform Wars

- Aplikace pracuje proti **skutečnému, uživatelem kontrolovanému cloudovému prostředí**.
- Vyhodnocení testuje **skutečný stav infrastruktury** po doběhnutí Terraformu, ne pouze odevzdaný kód.

Aktuální podpora a rozšiřitelnost

Zatím je implementována podpora pouze pro **GCP** poskytovatele. Architektura je navržena s důrazem na rozšiřitelnost o další poskytovatele (např. **AWS**) bez zásahu do jádra aplikace.



Architektura aplikace

Vrstvy architektury

- 1 **Frontend** (Angular SPA nasazená na Firebase Hosting)
- 2 **Backend** — (Django REST API nasazené jako Cloud Run služby)
- 3 **Izolované prostředí** — (Python CLI nasazené do Cloud Run Jobs)
- 4 **GCP služby** — Cloud SQL, Pub/Sub, Cloud Tasks, Cloud Scheduler





Veřejné API

Obsluhuje běžné požadavky klienta a deleguje náročné operace na pozadí.

Interní API

Zpracovává úlohy na pozadí. Jednotné rozhraní pro **Cloud Tasks, Pub/Sub a Cloud Scheduler**.

Events API

Asynchronní (ASGI), vystavuje **Server-Sent Events** endpointy pro real-time aktualizace.

Směrování mezi synchronní a asynchronní částí řídí aplikační load balancer. Interní API není z veřejné sítě dostupné.



Izolované prostředí — Cloud Run Jobs

Izolované prostředí pro běh Terraformu

- Každé odevzdání úlohy vyvolá spuštění **samostatného Cloud Run Jobu** ve vlastním kontejneru.
- Dočasný souborový systém — po doběhnutí se mažou **i lokální data** (state, plan, plugins).
- Vstup pro kontejner přes proměnné prostředí.
- Výstup publikován do Pub/Sub topicu.
- Běh Terraformu může trvat řádově minuty.

Dva oddělené Joby

`execute` (terraform init/plan/apply) a `validate` (validační Bash skript) — každý běží ve vlastním izolovaném kontejneru s minimem oprávnění.



Doručení výsledku odevzdání úlohy pomocí SSE

Problém

Výsledek odevzdání úlohy a jejího vyhodnocení musí dorazit **té instanci** Events API služby, která drží otevřené SSE spojení s konkrétním klientem — instance se ale mohou dynamicky škálovat.

Průběh doručení výsledku odevzdání úlohy

- 1 Cloud Run Job publikuje výsledek do **Pub/Sub topicu**.
- 2 Každá instance Events API si při startu vytvoří **vlastní pull subscription**.
- 3 Příchozí zpráva se předá z vlákna do **asyncio fronty**.
- 4 **SSE** stream pošle událost prohlížeči klienta.



Propojení s projektem Terraform poskytovatele

2 Select or setup new project → Projekt Bilbosti 3

Select existing project or create a new one. If you decide to create a new project, hit reload projects button below to refresh the list and wait for the new project to appear.

Instructions for configuring a project in Google Cloud Platform

Setup instructions	Manual	
Setup script	Automated	

Select project [Reload projects](#)

	Project ID	Project Number	Display Name	Parent
	terraform-wars-1	projects/130622077556	terraform-wars-1	
☒	terraform-wars-2	projects/964217697629	Terraform Wars 2	
☐	peerless-garage-457315-u0	projects/869770618702	terraform-wars	
☐	niam2-oauth2-demo	projects/698770600455	niam2-oauth2-demo	

[Back](#) [Continue](#)

- Uživatel vybere Terraform poskytovatele a dle instrukcí projekt nakonfiguruje.
- Aplikace ověří konfiguraci projektu na pozadí a o výsledku uživatele notifikuje.



Autentizace Terraformu vůči GCP projektu

Autentizace vůči uživatelově GCP projektu

- Uživatel ve svém projektu vytvoří nový SA s rolí **Owner**.
- Uživatel udělí aplikačnímu SA roli **Service Account Token Creator**.
- Aplikace je schopná vytvořit dočasný token a při komunikaci s GCP vystupovat pod identitou uživatele SA.

Důležité

Aplikace **nikam neukládá** přístupové údaje ke cloudovému projektu uživatele.



Odevzdání úlohy

The screenshot shows the Terraform Web IDE interface. At the top, there are tabs for 'Terraform Web', 'Tutorials', and 'Submissions'. Below the tabs, there's a header 'Create VPC network' with a 'Back' button and two buttons: 'Browse submissions' and 'Start new submission'. The main content area is divided into two panes: 'Instructions' on the left and 'Solution' on the right.

Instructions Pane:

- Create VPC network**
- Prerequisites**
 - 1. An active GCP Project ID.
- Step 1: Configure the provider**

Replace "YOUR_PROJECT_ID" with your selected GCP Project ID

```
provider "google" {  
  project = "YOUR_PROJECT_ID"  
  region  = "europe-west3"  
}
```
- Step 2: Enable Compute engine API**

```
resource "google_project_service" "compute_api" {  
  service = "compute.googleapis.com"  
  disable_on_destroy = true  
}
```

Solution Pane:

```
1 terraform {  
2   required_providers {  
3     google = {  
4       source = "hashicorp/google"  
5       version = "6.6.0"  
6     }  
7   }  
8 }  
9  
10 provider "google" {  
11   project = "dp-opponent-test-project-0"  
12   region  = "europe-west3"  
13 }  
14  
15 resource "google_project_service" "compute_api" {  
16   service = "compute.googleapis.com"  
17   disable_on_destroy = true  
18 }  
19  
20  
21 resource "google_compute_network" "vpc_network" {  
22   name = "terraform-ws-network"  
23   auto_create_subnetworks = true  
24 }
```

At the bottom of the Solution pane, there's a 'Provider project' dropdown menu with the value 'dp-opponent-test-project-1 (dp-opponent-test-...)' and a 'Choose where Terraform will provision the resources.' label. There are 'Next' and 'Submit' buttons at the bottom right.

- Split-screen rozhraní.
- V levé části obrazovky je zadání úlohy a následně výsledek odevzdaného řešení.
- V pravé části Monaco editor se zvýrazňováním syntaxe HCL.



Zobrazení výsledků odevzdání úlohy

Instructions

Output

Overall solution status: **FAILED**

 Please note, that the evaluation process may take a while to complete. You will be notified when the evaluation is complete.

 **Run your solution** 

 **Evaluate your solution** 

```
WARNING: [terraformwars-validator-sa@terraformwars.iam.gserviceaccount.com] does not have permission
to access projects instance [dp-oponent-test-project-3] (or it may not exist): The caller does not
have permission. This command is authenticated as terraformwars-validator-
sa@terraformwars.iam.gserviceaccount.com which is the active account specified by the [core/account]
property
Updated property [core/project].
Updated property [auth/impersonate_service_account].
Chyba: Secret 'test' nebyl nalezen!
```

- Stav odevzdání se v reálném čase aktualizuje **v reálném čase** pomocí SSE.
- Uživatel může stránku kdykoliv opustit a vrátit se. O výsledku odevzdání je informován prostřednictvím systémové notifikace.



Shrnutí

- Provedena analýza cloudových poskytovatelů a existujících platform pro výuku Terraformu.
- Navržena architektura aplikace s důrazem na **bezpečnost, škálovatelnost a izolaci** běhu úloh.
- Aplikace byla implementována a je nasazena v testovacím provozu na app.terraformwars.com.
- Zdrojový kód práce publikován pod MIT licenci.

Budoucí rozvoj

Rozšíření podpory o další Terraform poskytovatele (AWS, Azure, ...).

Děkuji za pozornost



Otázka 1

Otázka

Jak je v izolovaném prostředí Cloud Run Jobs řešena persistence state souboru? Ukládá se state po každém odevzdání, nebo se zahazuje?

Odpověď

- Aplikace sama state nepersistuje.
- Původní myšlenka byla state ukládat na Cloud Storage, od toho jsme upustili.
- Cílem je, aby si autor úlohy mohl nastavit Terraform tak, aby se state ukládal např. do uživatelova vlastního bucketu.
- Otevírá to prostor pro navazující úlohy, které mezi sebou potřebují sdílet state.



Otázka 2

Otázka

Co se stane s cloudovými zdroji vytvořenými uživatelem po neúspěšném odevzdání nebo po skončení testování? Je implementován mechanismus automatického terraform destroy?

Odpověď

- Automatický destroy není implementován.
- Původní myšlenka byla destroy neimplementovat kvůli navazujícím úlohám.
- Prostor pro budoucí rozvoj.



Otázka 3

Otázka

Jaká jsou bezpečnostní rizika spojená s tím, že validační skript je psán v Bash a může volat libovolné CLI příkazy?

Odpověď

- Vytvořené úlohy procházejí schvalovacím procesem — žádná úloha není veřejná bez ručního review.
- Validační skript běží v izolovaném prostředí pod SA, jehož jediné oprávnění je publikace do Pub/Sub topicu výsledků.
- Nemá přístup k databázi aplikace ani jiným uživatelům — pracuje jen nad GCP projektem řešitele.